BRAC University – CSE 221: Algorithms

Mid Semester Exam, Fall 2022 — Complete Solutions (Set A & Set B)

SET A

Question 1: Sorting Jack's List [10 Marks]

Given array (n=8):

Index	0	1	2	3	4	5	6	7
Number	23	2	19	3	7	11	5	13

Part 1: O(n²) Sort — Insertion Sort [3 marks]

Algorithm chosen: Insertion Sort — scans each element and inserts it into its correct position among previously sorted elements. Time complexity $O(n^2)$ in the worst case.

Step-by-step execution:

Pass	Array State	Action
Start	[23, 2, 19, 3, 7, 11, 5, 13]	Initial array
i=1	[2, 23, 19, 3, 7, 11, 5, 13]	2 < 23 → shift 23 right, insert 2 at index 0
i=2	[2, 19, 23, 3, 7, 11, 5, 13]	19 < 23 → shift 23, insert 19 at index 1
i=3	[2, 3, 19, 23, 7, 11, 5, 13]	3 < 19, 3 < 23 → shifts; insert 3 at index 1
i=4	[2, 3, 7, 19, 23, 11, 5, 13]	7 inserted at index 2
i=5	[2, 3, 7, 11, 19, 23, 5, 13]	11 inserted at index 3
i=6	[2, 3, 5, 7, 11, 19, 23, 13]	5 inserted at index 2
i=7	[2, 3, 5, 7, 11, 13, 19, 23]	13 inserted at index 5
Done	[2, 3, 5, 7, 11, 13, 19, 23]	Final sorted array ✓

Part 2: Jack's Linear Time Sort [4 marks]

Key Insight: The array already has two hidden sorted subsequences:

- **Even indices (0,2,4,6):** 23, 19, 7, 5 — in *decreasing* order
- **Odd indices (1,3,5,7):** 2, 3, 11, 13 — in *increasing* order

Jack uses a variation of **Merge Sort's merge step** — treating the two subsequences as two already-sorted lists and merging them in **O(n)** time.

Step-by-step Linear Merge:

Extract subsequences:

- Even-indexed (reversed to get ascending): [5, 7, 19, 23]
- Odd-indexed: [2, 3, 11, 13]

Now merge two sorted arrays:

Step	Even ptr → val	Odd ptr → val	Merged so far	Action
1	5	2	[2]	2 < 5 → take 2 from odd
2	5	3	[2,3]	3 < 5 → take 3 from odd

3	5	11	[2,3,5]	5 < 11 → take 5 from even
4	7	11	[2,3,5,7]	7 < 11 → take 7 from even
5	19	11	[2,3,5,7,11]	11 < 19 → take 11 from odd
6	19	13	[2,3,5,7,11,13]	13 < 19 → take 13 from odd
7	19	—	[2,3,5,7,11,13,19]	odd exhausted, take 19
8	23	—	[2,3,5,7,11,13,19,23]	take 23 → Done ✓

Time Complexity: Extracting two subsequences = O(n). Merging two sorted lists of n/2 each = O(n). Total = **O(n) ✓**

Part 3: Finding Index for 15 using Binary Search [3 marks]

After sorting, the list is: **[2, 3, 5, 7, 11, 13, 19, 23]**.
We use **Binary Search** to find the correct position to insert 15. Binary search runs in **O(log n)** — far more efficient than linear scan O(n).

Step	Low	High	Mid	arr[Mid]	Decision
1	0	7	3	7	15 > 7 → search right half
2	4	7	5	13	15 > 13 → search right half
3	6	7	6	19	15 < 19 → search left half
4	6	5	—	—	low > high → STOP
Result	—	—	—	—	Insert at index 6 (between 13 and 19) ✓

Resulting list after insertion: [2, 3, 5, 7, 11, 13, **15**, 19, 23]
Time complexity of binary search: **O(log n)**

Question 2: Graph Theory [10 Marks] — Set A

Graph edges (from figure): S–A, S–B, A–C, A–D, B–D, B–E, C–F, C–D, D–E, E–G, F–H, G–H
Vertices: S, A, B, C, D, E, F, G, H | **Total edges m = 12**

Part a: Justify Σ deg(v) = 2m [2 marks]

Every edge (u, v) contributes exactly **1** to deg(u) and **1** to deg(v). Therefore each edge is counted **twice** in the total degree sum.

Vertex	Neighbours	Degree
S	A, B	2
A	S, C, D	3
B	S, D, E	3
C	A, F, D	3
D	A, B, C, E	4
E	B, D, G	3
F	C, H	2
G	E, H	2
H	F, G	2
	Σ deg(v)	**24 = 2×12 = 2m ✓**

Part b: Adjacency Matrix (Set A — DFS) [2 marks]

	S	A	B	C	D	E	F	G	H
S	0	1	1	0	0	0	0	0	0
A	1	0	0	1	1	0	0	0	0
B	1	0	0	0	1	1	0	0	0
C	0	1	0	0	1	0	1	0	0
D	0	1	1	1	0	1	0	0	0
E	0	0	1	0	1	0	0	1	0
F	0	0	0	1	0	0	0	0	1
G	0	0	0	0	0	1	0	0	1
H	0	0	0	0	0	0	1	1	0

Part c: Depth First Search (DFS) from S [6 marks]

DFS explores as deep as possible before backtracking. Uses a stack (or recursion). Vertices are marked visited when first encountered.

DFS Traversal steps (assuming alphabetical neighbour order):

Step	Visit	Stack (top→)	Visited Set	Action
1	S	S	S	Start; push S
2	A	A,S	S,A	Pop S; push unvisited neighbours B,A (A on top)
3	C	C,A,S	S,A,C	Pop A; push unvisited D,C (C on top)
4	D	D,C,S	S,A,C,D	Pop C; push F,D (D on top); F also pushed
5	B	B,D,C,S	S,A,C,D,B	Pop D; push E,B
6	E	E,B	S,A,C,D,B,E	Pop B; E not yet visited
7	G	G,E	S,A,C,D,B,E,G	Pop E; push G
8	H	H	S,A,C,D,B,E,G,H	Pop G; push H
9	F	F	S,A,C,D,B,E,G,H,F	Pop H; push F (already in stack)
Done	—	Empty	S,A,C,D,B,E,G,H,F	All vertices visited ✓

DFS Order: S → A → C → D → B → E → G → H → F
DFS Tree edges: S–A, A–C, C–D, D–B, B–E, E–G, G–H, H–F
Time complexity: O(V + E) = O(9 + 12) = O(21)

Question 3: Time Complexity [10 Marks] — Set A

Part a: Finding_Worst_Case(n) [CO7]

Code analysis:

Outer loop (i): i = n; i /= 7 each iteration → i runs as n, n/7, n/49, … until i < 1
Number of iterations = $\log_7(n)$ = **O(log n)**

Second loop (j): j = 1 to n, step +3 → runs n/3 times = **O(n)**

Inner loop (k): k = 1 to 40, step +1 → always **40 iterations = O(1)** (constant, independent of n)

Inner loop (m): m = n down to 1, step –5 → runs n/5 times = **O(n)**

Total complexity calculation:

T(n) = [outer i loop] × [j loop] × ([k loop] + [m loop])
T(n) = O(log n) × O(n) × (O(1) + O(n))
T(n) = O(log n) × O(n) × O(n)
T(n) = $O(n^2 \log n)$

Part b: Recurrence T(n) = T(n/2) + T(n/4) + n [CO7]

We use the **Recursion Tree Method**.

> **Level 0:** cost = n
> **Level 1:** T(n/2) + T(n/4) → costs n/2 + n/4 = 3n/4
> **Level 2:** T(n/4)+T(n/8)+T(n/8)+T(n/16) → cost = n/4+n/8+n/8+n/16 = (4+2+2+1)n/16 = 9n/16
> **Pattern:** each level's cost = $(3/4)^k$ × n at level k

Total cost = n × $\Sigma(3/4)^k$ for k = 0 to ∞
= n × 1/(1 – 3/4) [geometric series with ratio 3/4 < 1]
= n × 4
T(n) = O(n)

> The recursion branches with ratio 3/4 each level, which is a convergent geometric series. The dominant cost is at the root, giving linear time.

SET B

Question 1: Sorting Jack's List [10 Marks] — Set B

Given array (n=8):

Index	0	1	2	3	4	5	6	7
Number	33	7	29	11	17	23	13	31

Part 1: O(n²) Sort — Insertion Sort [3 marks]

Pass	Array State	Action
Start	[33, 7, 29, 11, 17, 23, 13, 31]	Initial
i=1	[7, 33, 29, 11, 17, 23, 13, 31]	7 < 33 → insert 7 at index 0
i=2	[7, 29, 33, 11, 17, 23, 13, 31]	29 < 33 → insert 29 at index 1
i=3	[7, 11, 29, 33, 17, 23, 13, 31]	11 inserted at index 1
i=4	[7, 11, 17, 29, 33, 23, 13, 31]	17 inserted at index 2
i=5	[7, 11, 17, 23, 29, 33, 13, 31]	23 inserted at index 3
i=6	[7, 11, 13, 17, 23, 29, 33, 31]	13 inserted at index 2
i=7	[7, 11, 13, 17, 23, 29, 31, 33]	31 inserted at index 6
Done	[7, 11, 13, 17, 23, 29, 31, 33]	Sorted ✓

Part 2: Jack's Linear Time Sort [4 marks]

Key Insight: The array has two hidden sorted subsequences:

- **Even indices (0,2,4,6):** 33, 29, 17, 13 — *decreasing* → reverse = [13,17,29,33] ascending
- **Odd indices (1,3,5,7):** 7, 11, 23, 31 — already *increasing*

Step	Even val	Odd val	Merged so far	Action
1	13	7	[7]	7 < 13 → take odd
2	13	11	[7,11]	11 < 13 → take odd
3	13	23	[7,11,13]	13 < 23 → take even
4	17	23	[7,11,13,17]	17 < 23 → take even
5	29	23	[7,11,13,17,23]	23 < 29 → take odd
6	29	31	[7,11,13,17,23,29]	29 < 31 → take even
7	33	31	[7,11,13,17,23,29,31]	31 < 33 → take odd
8	33	—	[7,11,13,17,23,29,31,33]	take 33 → Done ✓

Time Complexity: O(n) — one linear scan to merge two sorted halves.

Part 3: Finding Index for 12 using Binary Search [3 marks]

Sorted list: **[7, 11, 13, 17, 23, 29, 31, 33]**. Insert **12**.

Step	Low	High	Mid	arr[Mid]	Decision
1	0	7	3	17	12 < 17 → go left

2	0	2	1	11	12 > 11 → go right
3	2	2	2	13	12 < 13 → go left
4	2	1	—	—	low > high → STOP
Result	—	—	—	—	Insert at index 2 (between 11 and 13) ✓

Result: [7, 11, **12**, 13, 17, 23, 29, 31, 33] — 12 inserted at index 2.

Question 2: Graph Theory [10 Marks] — Set B

Same graph as Set A. Vertices: S, A, B, C, D, E, F, G, H | Edges: m = 12

Part a: Justify Σ deg(v) = 2m [2 marks]

Every undirected edge (u,v) contributes +1 to deg(u) and +1 to deg(v). Hence every edge is counted exactly **twice** in Σ deg(v). With m=12 edges and sum of degrees = 24: **24 = 2 × 12 ✓**

Part b: Adjacency List [2 marks]

Vertex	Adjacency List
S	→ A → B
A	→ S → C → D
B	→ S → D → E
C	→ A → D → F
D	→ A → B → C → E
E	→ B → D → G
F	→ C → H
G	→ E → H
H	→ F → G

Part c: Breadth First Search (BFS) from S [6 marks]

BFS explores all neighbours of a vertex before going deeper. Uses a **Queue** (FIFO).

Step	Dequeue	Queue (front→back)	Visited	Enqueued
1	S	A, B	S	Enqueue A, B
2	A	B, C, D	S,A	Enqueue C, D (S already visited)
3	B	C, D, E	S,A,B	Enqueue E (S,D already queued/visited)
4	C	D, E, F	S,A,B,C	Enqueue F (A,D already handled)
5	D	E, F	S,A,B,C,D	A,B,C,E already visited/queued
6	E	F, G	S,A,B,C,D,E	Enqueue G (B,D visited)
7	F	G, H	S,A,B,C,D,E,F	Enqueue H (C visited)
8	G	H	S,A,B,C,D,E,F,G	E,H — enqueue H if not yet
9	H	Empty	S,A,B,C,D,E,F,G,H	All visited ✓

BFS Order: S → A → B → C → D → E → F → G → H
BFS Tree edges: S–A, S–B, A–C, A–D, B–E, C–F, E–G, F–H

Time complexity: O(V + E) = O(9 + 12) = O(21)

Question 3: Time Complexity [10 Marks] — Set B

Part a: Finding_Worst_Case(n) [CO7]

Loop analysis:

- **Outer loop (i):** i = n; i -= 4 → runs from n down to 1 in steps of 4 → **O(n)** iterations
- **Second loop (j):** j = 1; j *= 3 → j runs 1, 3, 9, 27, … up to n → **$O(\log_3 n) = O(\log n)$**
- **Inner loop (k):** k = 1 to 20, step +1 → always **20 iterations = O(1)** (constant)
- **Inner loop (m):** m = n; m /= 5 → m runs n, n/5, n/25, … → **$O(\log_5 n) = O(\log n)$**

Total complexity calculation:

T(n) = [i loop] × [j loop] × ([k loop] + [m loop])
T(n) = O(n) × O(log n) × (O(1) + O(log n))
T(n) = O(n) × O(log n) × O(log n)
$T(n) = O(n \log^2 n)$

Part b: Recurrence T(n) = T(n/3) + T(n/5) + n [CO7]

Using the **Recursion Tree Method**:

- **Level 0:** cost = n
- **Level 1:** T(n/3) + T(n/5) → cost = n/3 + n/5 = 8n/15
- **Level 2:** costs = n/9 + n/15 + n/15 + n/25 = (25+15+15+9)n/225 = 64n/225
- **Pattern:** ratio per level = 8/15 < 1 (convergent geometric series)

Total = $n \times \Sigma(8/15)^k$ for k = 0,1,2,…
= n × 1/(1 – 8/15) [geometric series, ratio 8/15 < 1]
= n × 15/7
T(n) = O(n)

> Since 1/3 + 1/5 = 8/15 < 1, the branching factor is less than 1, meaning the work decreases geometrically at each level, and the root dominates → linear time.

Summary of Answers

Question	Set A Answer	Set B Answer
Q1.1 – $O(n^2)$ Sort	Insertion Sort → [2,3,5,7,11,13,19,23]	Insertion Sort → [7,11,13,17,23,29,31,33]
Q1.2 – Linear Sort	Merge even (rev) & odd subsequences O(n)	Merge even (rev) & odd subsequences O(n)
Q1.3 – Insert	Binary Search; 15 goes at index 6	Binary Search; 12 goes at index 2
Q2a – Σdeg=2m	Verified: 24 = 2×12 ✓	Verified: 24 = 2×12 ✓
Q2b – Representation	Adjacency Matrix	Adjacency List
Q2c – Traversal	DFS: S→A→C→D→B→E→G→H→F	BFS: S→A→B→C→D→E→F→G→H
Q3a – Loop Complexity	$O(n^2 \log n)$	$O(n \log^2 n)$
Q3b – Recurrence	T(n)=T(n/2)+T(n/4)+n → O(n)	T(n)=T(n/3)+T(n/5)+n → O(n)